COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE-010 Data Structures and Analysis	**SECTION:**	CPE21S1
DATE PERFORMED:	9/15/2026	**DATE SUBMITTED:**	9/15/2026
NAME:	Alvarez, Joshua Caleb P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 **Hands-on Activity 8.1 Sorting Algorithm Pt2**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of shell, merge and quick sort

Code + Console Screenshot

Sorting arlgo8P2 > main.cpp > ...

```cpp
//Alvarez, Joshua Caleb P.
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingp.h"

const int maxSize = 100;

void displayAll(int arr[]);
void copyArr(int source[], int destination[]);

int main() {
    int unsortedArr[maxSize];
    int copiedArr[maxSize];
    srand(time(0));
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand() % 100;
    }

    std::cout << "Unsorted Array: ";
    displayAll(unsortedArr);
```

```
Unsorted Array: 95 1 6 95 6 59 77 71 6 33 90 70 72 55 19 32 1 74 34 13 98 26 93 25 4
35 96 47 97 33 75 23 15 70 0 65 18 14 60 88 4 86 89 95 38 91 57 62 45 35 26 70 1 37
8 18 45 77 54 91 28 83 52 60 2 54 86 69 1 91 65 61 33 71 94 38 5 27 85 64
73 20 40 84 88 93 38 2 22 44 20 93 4 80 2 15 69 73 34 82 83
```

Observations

The program I created is an array with 100 randomly generated values. The values were presented in their natural order and had not been sorted in ascending order. This non-sorted array was employed as the initial set of data for sorting via Shell, Merge and Quick Sort algorithms to make sure the comparison was valid.

Table 8-1. Array of Values for Sort Algorithm Testing

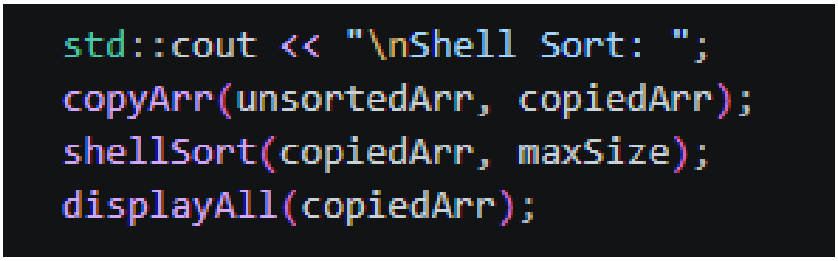
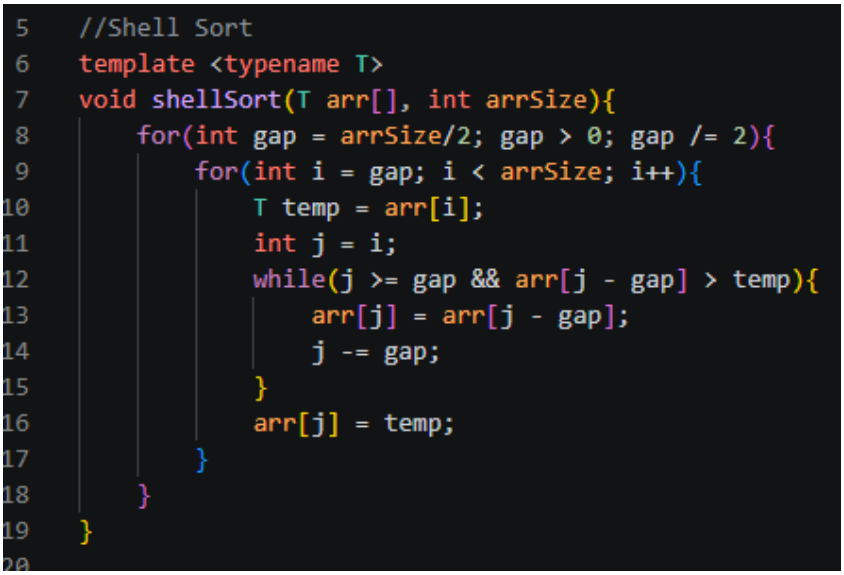
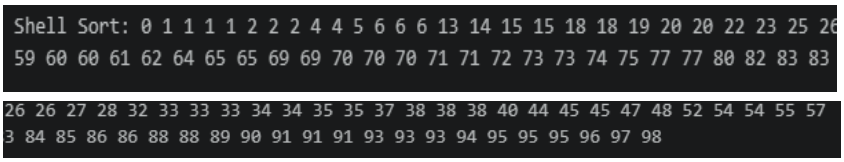
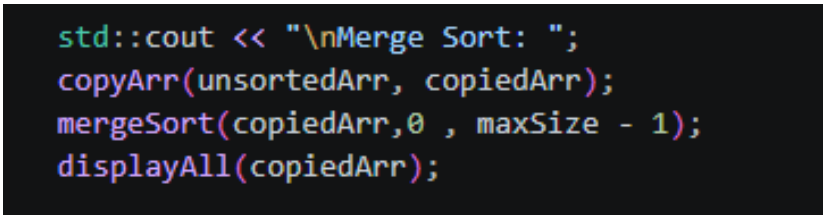
Code + Console Screenshot	  
Observations	The Shell Sort technique has been successful in organizing the 100 randomly generated numbers in an ascending manner. This algorithm involved comparing numbers that were a certain gap apart and then gradually decreasing the gap until finally, when the gap was decreased to a certain level, the numbers were arranged in the right places.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	

```cpp
//merge sort helper
template <typename T>
void merge(T arr[], int left, int mid, int right){
    int n1 = mid - left + 1;
    int n2 = right - mid;

    T* leftArr = new T[n1];
    T* rightArr = new T[n2];

    for(int i = 0; i < n1; i++){
        leftArr[i] = arr[left + i];
    }
    for(int j = 0; j < n2; j++){
        rightArr[j] = arr[mid + 1 + j];
    }

    int i = 0 , j = 0, k = left;
    while(i < n1 && j < n2){
        if(leftArr[i] <= rightArr[j]){
            arr[k] = leftArr[i];
            i++;
        }
        else{
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }
```

```
    while(i < n1){
        arr[k] = leftArr[i];
        i++;
        k++;
    }
    while(j < n2){
        arr[k] = rightArr[j];
        j++;
        k++;
    }
}

//merge sort
template <typename T>
void mergeSort(T arr[], int left, int right){
    if(left < right){
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

```
Merge Sort: 0 1 1 1 1 2 2 2 4 4 5 6 6 6 13 14 15 15 18 18 19 20 20 22 23 25 26 26 27
 59 60 60 61 62 64 65 65 69 69 70 70 70 71 71 72 73 73 74 75 77 77 80 82 83 83 84 85
```

```
27 28 32 33 33 33 34 34 35 35 37 38 38 38 40 44 45 45 47 48 52 54 54 55 57
5 86 86 88 88 89 90 91 91 91 93 93 93 94 95 95 95 96 97 98
```

Observations	Merge sort algorithm has been able to sort the array in increasing order starting from the smallest number to the largest number. The sorting process involved breaking down the array into smaller arrays and merging them until only one sorted array is formed.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	

```
    std::cout << "\nQuick Sort: ";
    copyArr(unsortedArr, copiedArr);
    quickSort(copiedArr,0 , maxSize - 1);
    displayAll(copiedArr);


    return 0;
}


void displayAll(int arr[]){
    for(int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

void copyArr(int source[], int destination[]){
    for(int i = 0; i < maxSize; i++){
        destination[i] = source[i];
    }
}
```

```cpp
//quicksort Helper
template <typename T>
int partition(T arr[], int low, int high){
    T pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++){
        if(arr[j] <= pivot){
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i+1], arr[high]);
    return i + 1;
}

template <typename T>
void quickSort(T arr[], int low, int high){
    if(low < high){
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex -1);
        quickSort(arr, pivotIndex+1, high);
    }
}
```

```
Quick Sort: 0 1 1 1 1 2 2 2 4 4 5 6 6 6 13 14 15 15 18 18 19 20 20 22 23 25 26 26 27
59 60 60 61 62 64 65 65 69 69 70 70 70 71 71 72 73 73 74 75 77 77 80 82 83 83 84 85
```

```
6 27 28 32 33 33 33 34 34 35 35 37 38 38 38 40 44 45 45 47 48 52 54 54 55 57
85 86 86 88 88 89 90 91 91 91 93 93 93 94 95 95 95 96 97 98
```

Observations	Quick sort effectively sorted all 100 numbers in increasing order. The quick sort algorithm used a pivot element and sorted the array according to the comparison of the other elements with the pivot element. The process was continued until all the smaller arrays were sorted.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

PROBLEM 1: Main.cpp

```cpp
#include <iostream>
#include "sortingAlgorithms.h"

void display(int arr[], int size);

int main() {

    // Example array after Quick Sort partition
    int leftSubList[] = {3, 4, 2};
    int rightSubList[] = {8, 7, 9};

    int leftSize = 3;
    int rightSize = 3;

    std::cout << "QUICK SORT PARTITION EXAMPLE\n";
    std::cout << "============================\n";

    std::cout << "Left Sub-list: ";
    display(leftSubList, leftSize);

    std::cout << "Pivot: 6\n";

    std::cout << "Right Sub-list: ";
    display(rightSubList, rightSize);

    // Sort the left and right sub-lists
    // using Insertion Sort
    insertionSort(leftSubList, leftSize);
    insertionSort(rightSubList, rightSize);

```

```
31      std::cout << "\nAFTER USING INSERTION SORT\n";
32      std::cout << "==========================\n";
33
34      std::cout << "Left Sub-list: ";
35      display(leftSubList, leftSize);
36
37      std::cout << "Pivot: 6\n";
38
39      std::cout << "Right Sub-list: ";
40      display(rightSubList, rightSize);
41
42      std::cout << "\nFinal Sorted Array: ";
43
44      display(leftSubList, leftSize);
45      std::cout << "6 ";
46      display(rightSubList, rightSize);
47
48      return 0;
49  }
50
51  void display(int arr[], int size) {
52      for (int i = 0; i < size; i++) {
53          std::cout << arr[i] << " ";
54      }
55
56      std::cout << std::endl;
57  }
```

sortingAlgorithms.h

```cpp
#ifndef SORTINGALGORITHMS_H
#define SORTINGALGORITHMS_H

// Shell Sort
template <typename T>
void shellSort(T arr[], int arrSize) {
    for (int gap = arrSize / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < arrSize; i++) {
            T temp = arr[i];
            int j = i;

            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }

            arr[j] = temp;
        }
    }
}

// Merge Sort Helper
template <typename T>
void merge(T arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    T* leftArr = new T[n1];
    T* rightArr = new T[n2];
```

```
30
31      for (int i = 0; i < n1; i++) {
32          leftArr[i] = arr[left + i];
33      }
34
35      for (int j = 0; j < n2; j++) {
36          rightArr[j] = arr[mid + 1 + j];
37      }
38
39      int i = 0, j = 0, k = left;
40
41      while (i < n1 && j < n2) {
42          if (leftArr[i] <= rightArr[j]) {
43              arr[k] = leftArr[i];
44              i++;
45          } else {
46              arr[k] = rightArr[j];
47              j++;
48          }
49          k++;
50      }
51
52      while (i < n1) {
53          arr[k] = leftArr[i];
54          i++;
55          k++;
56      }
```

```
    while (j < n2) {
        arr[k] = rightArr[j];
        j++;
        k++;
    }

    delete[] leftArr;
    delete[] rightArr;
}

// Merge Sort
template <typename T>
void mergeSort(T arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

```cpp
// Quick Sort Helper
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    std::swap(arr[i + 1], arr[high]);

    return i + 1;
}

// Quick Sort
template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);

        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
```

```cpp
// Quick Sort
template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);

        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}

// Insertion Sort
template <typename T>
void insertionSort(T arr[], int size) {
    for (int i = 1; i < size; i++) {
        T key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

#endif
```

OUTPUT

```
QUICK SORT PARTITION EXAMPLE
============================
Left Sub-list: 3 4 2
Pivot: 6
Right Sub-list: 8 7 9
```

```
AFTER USING INSERTION SORT
==========================
Left Sub-list: 2 3 4
Pivot: 6
Right Sub-list: 7 8 9

Final Sorted Array: 2 3 4
6 7 8 9
```

Problem 2: Main.cpp

```cpp
//Alvarez, Joshua Caleb P.
#include <iostream>
#include "quickSort.h"

const int SIZE = 15;

void displayArray(int arr[])
{
    for (int i = 0; i < SIZE; i++)
    {
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;
}

int main()
{
    int arr[SIZE] = {
        4, 34, 29, 48, 53,
        87, 12, 30, 44, 25,
        93, 67, 43, 19, 74
    };

    std::cout << "Original Array: ";
    displayArray(arr);

    quickSort(arr, 0, SIZE - 1);

    std::cout << "Sorted Array: ";
    displayArray(arr);

    std::cout << "\nQuick Sort Average Time Complexity: O(N log N)" << std::endl;
    std::cout << "Quick Sort Worst Time Complexity: O(N^2)" << std::endl;

    return 0;
}
```

quickSort.h

SUPPLEMENTARY_P2 > C quickSort.h > ...

```
1   //Alvarez, Joshua Caleb P.
2   #ifndef QUICKSORT_H
3   #define QUICKSORT_H
4
5   void swapValues(int &a, int &b)
6   {
7       int temp = a;
8       a = b;
9       b = temp;
10  }
11
12  int partition(int arr[], int low, int high)
13  {
14      int pivot = arr[high];
15      int i = low - 1;
16
17      for (int j = low; j < high; j++)
18      {
19          if (arr[j] < pivot)
20          {
21              i++;
22              swapValues(arr[i], arr[j]);
23          }
24      }
25
26      swapValues(arr[i + 1], arr[high]);
27
28      return i + 1;
29  }
30
```

```
31  void quickSort(int arr[], int low, int high)
32  {
33      if (low < high)
34      {
35          int pivotIndex = partition(arr, low, high);
36
37          quickSort(arr, low, pivotIndex - 1);
38          quickSort(arr, pivotIndex + 1, high);
39      }
40  }
41
42  #endif
```

OUTPUT

```
[Running] cd "c:\Users\RYZEN 5\OneDrive\DSA\SUPPLEMENTARY_P2\" && g++
Original Array: 4 34 29 48 53 87 12 30 44 25 93 67 43 19 74
Sorted Array: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93

Quick Sort Average Time Complexity: O(N log N)
Quick Sort Worst Time Complexity: O(N^2)

[Done] exited with code=0 in 1.108 seconds
```

CONCLUSION: Summary of Lessons Learned

- In this activity, I learned how Shell Sort, Merge Sort, and Quick Sort operate and how they can be used to sort an unsorted array in increasing order. I came to learn that each of these sorting algorithms operates differently. For example, Shell Sort uses gaps, Merge Sort divides and merges arrays, and Quick Sort divides an array by a pivot. I learned how to make a header file for the sorting algorithms and link it to the main.cpp file. This activity made me understand more about how these sorting algorithms operate and how the difference approaches can affect their performance

Analysis of the Procedure:

- The procedure helped me understand sorting algorithms better because I was able to create an unsorted array and apply different sorting methods to it. By checking the output, I could see how each algorithm successfully arranged the values in ascending order.

Analysis of the Supplementary Activity:

- Through the supplementary task, I got to know that Quick Sort can also use other sorts to sort the left and right sublists. Further, I came to know that the average time complexity of Merge Sort and Quick Sort can be O(N log N).

Concluding Statement / Feedback:

- I think I did well in this activity because I was able to understand and implement the three sorting algorithms. My main area for improvement is understanding the steps of each algorithm more deeply, especially the partitioning and merging processes. I also need more practice writing the sorting functions without relying too much on examples.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt and grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of sorting algorithms, clarification of the activity instructions, and grammar improvement.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to assist in understanding the sorting algorithms and developing/organizing the C++ code. I reviewed and modified the provided code and used my own understanding and analysis to complete the activity. AI did not replace my original thinking, analysis, or authorship of the final work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.